Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

Kết luận **2**
Tổng quan quá trình chuyển đổi 2
Lộ trình chuyển đổi thực tế 3
Khi nào áp dụng — heuristic thực tế 3
Điều không thay đổi 4

Kết luận

> Kiến trúc không phải đích đến, mà là hành trình liên tục của những quyết định có cân nhắc và những đánh đổi có ý thức.
> — Tổng hợp từ Richardson [2a] và Newman [4a]

Sau 12 chương, chúng ta đã đi qua toàn bộ quá trình chuyển đổi của KBM LMS — từ ba monolith ban đầu đến kiến trúc microservices đang tiếp tục hoàn thiện. Chương kết này không giới thiệu pattern mới, mà tổng hợp các quyết định đã phân tích thành một **lộ trình thực tế có thứ tự ưu tiên**.

Tổng quan quá trình chuyển đổi

Nhìn lại, KBM đã áp dụng các nguyên lý của kiến trúc hướng dịch vụ (SOA) như tính tự chủ (Autonomy) và hợp đồng dịch vụ chuẩn hóa (Standardized Service Contract), nhưng tránh được điểm yếu của SOA truyền thống. Thay vì dùng Enterprise Service Bus (ESB) nặng nề, hệ thống đi theo nguyên tắc "smart endpoints, dumb pipes" của microservices (sử dụng API Gateway và Kafka).

Kiến trúc phần mềm giống như việc quy hoạch một khuôn viên trường đại học. Ban đầu, mọi thứ (phòng học, thư viện, giáo vụ) có thể xếp chung trong một tòa nhà đa năng (monolith). Nhưng khi quy mô tăng lên, chúng ta phải xây tách rời thành khu giảng đường, thư viện, ký túc xá (microservices), và cần quy hoạch hệ thống đường nội bộ, biển chỉ dẫn (API Gateway/Event Bus) để liên kết chúng.

Mỗi chương giải quyết một vấn đề cụ thể của KBM trong quá trình "quy hoạch" đó:

Bảng K.1: *Quá trình chuyển đổi của KBM qua 12 chương — vấn đề và giải pháp*

Chương	Vấn đề KBM gặp phải	Giải pháp (bài học của chương)
1	Ba monolith rời rạc: logic trùng lặp, danh tính không liên thông, chi phí bảo trì nhân lên	Quyết định chuyển đổi có cơ sở (khung ra quyết định)
2	Không rõ ranh giới service, thiết kế theo hướng thực thể (entity-driven design)	DDD + Event Storming → 7 Bounded Context core
3	API không nhất quán, breaking change thường xuyên	REST + OpenAPI + chiến lược versioning
4	Một service lỗi gây hiệu ứng dây chuyền (cascading failure)	Circuit Breaker + retry + timeout (Resilience4j)
5	Chế độ thi: hàng trăm lượt nộp mỗi phút, cần giữ thứ tự (ordering) và phát lại (replay)	Kafka event streaming + idempotent consumer

Chương	Vấn đề KBM gặp phải	Giải pháp (bài học của chương)
6	Luồng nộp bài treo ở PENDING khi `kbm-judge-sql` dừng đột ngột, cần tính nguyên tử (atomicity)	Choreography-based Saga + compensation + semantic lock + timeout
7	Database dùng chung giữa Core và Academic; bảng xếp hạng đọc nhiều	Database-per-service + CQRS cho bảng xếp hạng
8	Mỗi client, mỗi service tự kiểm tra danh tính — trùng lặp, dễ lệch chính sách	API Gateway tập trung: routing, xác minh JWT ở biên, giới hạn lưu lượng
9	Tại thời điểm khảo sát (đầu năm 2026): JWT HS256 dùng shared secret, token tồn tại quá lâu	`kbm-auth`: HS256 → RS256 + luân phiên refresh token
10	Không thể viết lại toàn bộ một lần (Big Bang rewrite) — hệ thống đang vận hành	Strangler Fig — tách dần từng service, mang lại giá trị tức thì
11	Lỗi xảy ra nhưng không biết tại đâu, tại sao	Structured logging + distributed tracing + cảnh báo theo SLO
12	Triển khai thủ công N service vừa rủi ro vừa chậm	Docker Compose + CI/CD tự động (GitLab CI) + kế hoạch rollback

Lộ trình chuyển đổi thực tế

Biết các pattern là chưa đủ; còn phải áp dụng đúng **thứ tự**. Không phải tất cả pattern đều có giá trị như nhau ở từng giai đoạn. Nguyên tắc xuyên suốt: **các cải tiến nhanh (Quick Wins) trước, thay đổi rủi ro cao sau** — mỗi giai đoạn phải mang lại giá trị ngay, không cần đợi hoàn thành toàn bộ.

Lộ trình chi tiết cho KBM — bốn giai đoạn *các cải tiến nhanh (Quick Wins) → nền tảng Observability → Resilience & CI/CD → phân tách cơ sở dữ liệu*, kèm bảng hành động theo tác động × nỗ lực và các sai lầm thường gặp — đã được trình bày đầy đủ ở **Chương 10** (§10.5, *Lộ trình chuyển đổi — 4 giai đoạn* sai lầm thường gặp ở §10.6). Điểm mấu chốt của thứ tự này: phải *nhìn thấy* được hệ thống (observability) và đảm bảo khả năng chịu lỗi (resilience) *trước khi* tách database — bước rủi ro nhất.

Giá trị trước, pattern sau

Không nên áp dụng CQRS chỉ vì nó hấp dẫn về mặt kỹ thuật. Hãy áp dụng khi truy vấn bảng xếp hạng làm chậm database dùng chung của Core và Academic. Mọi pattern trong sách này đều nhằm giải quyết một vấn đề cụ thể, không phải để bản vẽ kiến trúc trông có vẻ hoàn chỉnh.

Khi nào áp dụng — heuristic thực tế

Không phải hệ thống nào cũng cần mọi pattern trong 12 chương. Bảng dưới đây tóm tắt các tín hiệu thực tế (heuristic) để ra quyết định:

Bảng K.2: *Heuristic — khi nào áp dụng pattern nào*

Tín hiệu	Pattern cần xem xét	Chương tham khảo
Nhiều nhóm phải deploy chung và chờ nhau (Ch.1 §1.6, ma trận Ch.10)	Tách service theo Bounded Context	Ch.1, 10
Thời gian phản hồi tăng khi tải tăng đột biến	Circuit Breaker + tách rời bất đồng bộ	Ch.4, 5
Nhiều service cần cùng dữ liệu (JOIN xuyên service)	API Composition / CQRS	Ch.7
Giao dịch xuyên 2+ service	Saga + compensation	Ch.6
Không biết lỗi xảy ra ở đâu	Distributed tracing (`traceId`) + `requestId` / `correlationId`	Ch.11
Triển khai mất hơn 30 phút hoặc cần nhiều người	Tự động hóa CI/CD	Ch.12

Tín hiệu	Pattern cần xem xét	Chương tham khảo
Token bị lộ nhưng không thể thu hồi	JWT thời hạn ngắn + luân phiên refresh token	Ch.9

Điều không thay đổi

Công cụ thay đổi — Spring Boot hôm nay có thể khác với Spring Boot năm năm sau. Nhưng các nguyên tắc cốt lõi được học trong sách này không phụ thuộc vào framework cụ thể:

- **Kiến trúc hướng dịch vụ (SOA) là nền tảng**: dù công nghệ thay đổi (từ SOAP sang REST/gRPC, từ ESB sang Kafka), triết lý định nghĩa ranh giới dịch vụ và hợp đồng dịch vụ chuẩn hóa vẫn không đổi.
- **Conway's Law**: cấu trúc hệ thống phản ánh cấu trúc tổ chức — thay đổi bên này phải đi kèm thay đổi bên kia.
- **Failure as first-class citizen**: microservices không loại bỏ lỗi mà làm lỗi trở nên tường minh, vì vậy cần thiết kế cho tình huống lỗi ngay từ đầu.
- **Eventual consistency là trade-off, không phải khuyết điểm**: cần phân biệt khi nào hệ thống đòi hỏi strong consistency (điểm số, kết quả chấm bài) và khi nào có thể chấp nhận eventual consistency (bảng xếp hạng).
- **Observability trước tối ưu hóa**: không thể cải thiện những gì không đo được. Tracing và metric là công cụ tư duy, không chỉ là công cụ debug.
- **Monolith First khi phù hợp**: bộ công cụ đã học không phải lúc nào cũng nên áp dụng — sự phán đoán về *khi nào* quan trọng hơn kỹ năng *làm thế nào*.

KBM sau lộ trình: một kịch bản

Nếu KBM đi đúng lộ trình ở Chương 10: sau các cải tiến nhanh và nền tảng observability (Giai đoạn 1–2), nhóm phát triển nhìn thấy lỗi nằm ở đâu thay vì đoán qua log từng container. Khi có cơ chế chịu lỗi, dead letter topic và timeout cho saga nộp bài (Giai đoạn 3), không còn bài nộp nào treo ở PENDING. Đến khi tách schema và thay truy vấn chéo bằng API (Giai đoạn 4), Academic Service mới triển khai được độc lập với Core. Trong kỳ thi, dashboard cho thấy độ trễ nộp bài nằm trong SLO của chế độ thi (Chương 11). Đây là kết quả có thể đạt được khi áp dụng có trình tự và có phán đoán.

Quá trình học không kết thúc ở chương này. Lĩnh vực này tiếp tục phát triển — service mesh, platform engineering, AI-assisted observability — nhưng các công nghệ mới vẫn dựa trên nền tảng kiến trúc hướng dịch vụ và microservices đã học trong cuốn sách này: Bounded Context, cô lập lỗi và tự động hóa triển khai.